

On-Device Network Traffic Anomaly Detection Using TinyML

MLP301 – Machine Learning Principles

Assessment 3—Machine Learning Prototype

Name: Tia Darvell

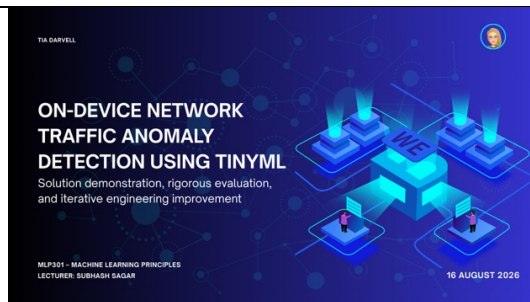
Student ID: A00029275

Torrens University Australia

Lecturer: Subhash Sagar

Submission Date: 16 August 2026

Speech



My name is Tia Darvell, and this presentation introduces a TinyML network anomaly detection prototype, built for the UNSW-NB15 dataset and deployed for live edge inference.

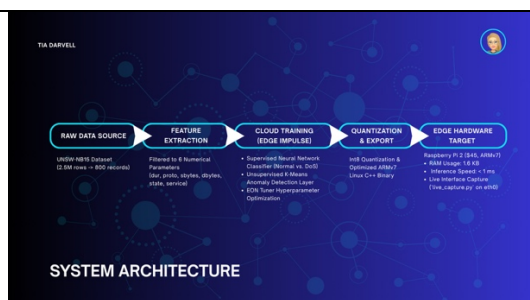
This demonstration illustrates how the prototype solves the latency and privacy vulnerabilities of cloud-based intrusion detection systems by bringing machine learning directly onto resource-constrained edge hardware.



The primary challenge addressed here is that intrusion detection usually assumes enterprise-grade hardware, which is often unaffordable for small networks.

Furthermore, traditional systems rely on static signatures, that struggle with novel zero-day threats, while routing telemetry to the cloud, this creates unacceptable latency for legacy architectures.

To determine if a viable machine learning pipeline can execute on older, budget-friendly hardware, this prototype was deployed on a 2014 Raspberry Pi 2, costing approximately \$45. This local execution bridges the gap by maintaining a memory footprint under two kilobytes while providing real-time security monitoring.



The system architecture follows a five-stage pipeline. Network traffic data from the UNSW-NB15 dataset is filtered and uploaded to Edge Impulse for cloud-based training.

Following feature reduction, the model is compiled into an optimized Linux ARMv7 binary tailored for quantized int8 execution. This binary is then deployed directly to the Raspberry Pi 2 for live, offline inference, ensuring resource allocation stays strictly within microcontroller limits without sacrificing classification throughput.

ML APPROACH (LAYERED DEFENCE)

Supervised Classification: Neural network trained to separate known Normal traffic from DoS attack signatures.

Unsupervised Anomaly Detection: K-means clustering layer to flag outlier data points that fall outside learned boundaries.

Why Both? Combines signature recognition with zero-day anomaly trapping.

The slide also features a small inset image of a user interface with various controls and a sidebar.

The machine learning approach utilizes a defense-in-depth strategy rather than a single classifier.

A neural network performs supervised classification between Normal and DoS traffic, handling known attack vectors. In parallel, an unsupervised K-means anomaly detection layer is implemented to catch structural outliers and attack patterns entirely absent from the training data.

This dual architecture ensures signature-based threats are intercepted while retaining the flexibility to flag unknown zero-day anomalies.

DATASET PARAMETERS
800 balanced records parsed from UNSW-NB15.

FEATURE REDUCTION
Filtered from 49 original attributes down to 6 key numerical parameters: dur, proto, sbytes, dbytes, state, and service.

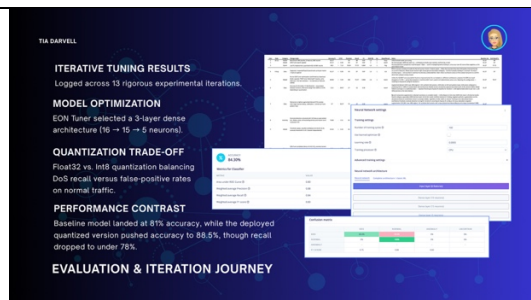
VALIDATION SPLIT
Stratified 81% training / 19% test split
(from 1619, held-out 312 cases)

DATASET & FEATURE EXTRACTION PIPELINE

The slide includes a small inset image of a user interface showing a list of records and a terminal window.

The training data utilizes a filtered slice of the UNSW-NB15 dataset, consisting of 800 records evenly split between Normal and DoS traffic. The feature space was reduced from 49 raw attributes down to 6 vital numerical parameters selected by information gain: duration, protocol, source bytes, destination bytes, connection state, and service type.

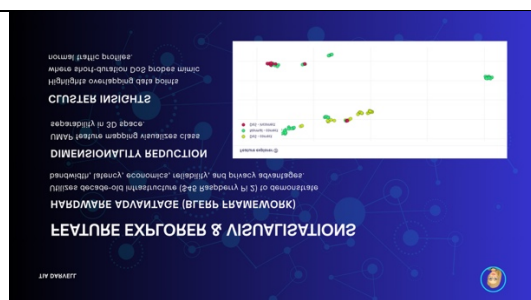
As demonstrated in the batch-inference video clip, pre-processed CSV files are evaluated against a 121-record held-out test split. This strict separation ensures baseline metrics reflect true unseen data performance rather than training bias.



The 13-stage iteration log, captured directly in the project datalog table, documents a rigorous scientific process. Baseline models achieved 81% accuracy, while early quantized versions deployed to the Pi pushed accuracy to 88.5%, though recall dropped to under 78%.

Because missing a DoS attack is the costliest failure mode, targeted data augmentation causing subsequent int8 models to collapse into high false-positive rates led to the active rejection of those pipelines.

Utilizing the EON Tuner, a 16-to-15-to-5 dense architecture was selected. Transitioning to int8 quantization created a deliberate security trade-off, increasing critical DoS recall at the expense of a minor rise in normal-class false alarms.



A defining characteristic of this project is the hardware target, utilizing decade-old infrastructure (\$45 Raspberry Pi 2) to demonstrate the BLERP framework advantages of bandwidth, latency, economics, reliability, and privacy.

Visualizing the feature space via UMAP dimensionality reduction yields profound insights. In this 3D scatter plot, green normal points cluster separately from yellow DoS instances.

However, the red misclassified points reveal where short-duration, low-byte DoS probes structurally overlap with legitimate traffic boundaries, explaining the exact performance ceilings.



Addressing project limitations is critical.

Cross-validation splits across held-out test data, proved a true DoS recall range of 60.4% to 75.0%, and false negatives remain the costliest failure mode.

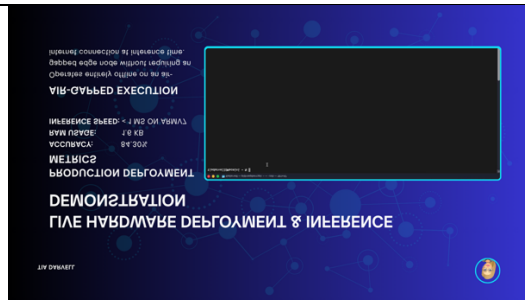
Additionally, reducing the dataset to six features, conflates different DoS attack sub-types.

As misclassified samples clustered together in the feature space in a way that suggests, fundamentally different attack patterns that the current model cannot distinguish.

This indicates that feature reduction, inherently introduces a separability ceiling that threshold tuning alone cannot fix.

Furthermore, automated optimization tools like the EON Tuner proved non-deterministic, yielding different results each time and demonstrating that automated hyperparameter search requires manual verification.

To mitigate feature conflation in subsequent iterations, secondary packet-header fields or temporal sliding windows should be incorporated into the feature extraction pipeline.



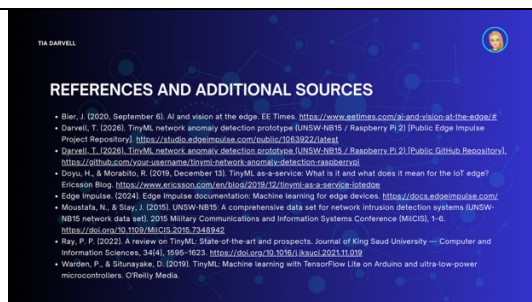
As captured in the live hardware deployment video on the Raspberry Pi 2, a Python wrapper binds directly to the network interface, processes real-time flow packets, and leverages the compiled C++ model.

The system operates entirely offline on an air-gapped edge node without requiring an internet connection at inference time.

Live classifications can be observed differentiating normal multicast traffic from structural anomalies on the fly.

Overall, the final deployment achieves 84.30% accuracy with sub-millisecond inference and a 1.6-kilobyte RAM footprint.

Thank you.



References

- Bier, J. (2020, September 6). AI and vision at the edge. EE Times.
<https://www.eetimes.com/ai-and-vision-at-the-edge/#>
- Darvell, T. (2026). TinyML network anomaly detection prototype (UNSW-NB15 / Raspberry Pi 2) [Public Edge Impulse Project Repository].
<https://studio.edgeimpulse.com/public/1063922/latest>
- Darvell, T. (2026). TinyML network anomaly detection prototype (UNSW-NB15 / Raspberry Pi 2) [Public GitHub Repository]. <https://github.com/your-username/tinyml-network-anomaly-detection-raspberrypi>
- Doyu, H., & Morabito, R. (2019, December 13). TinyML as-a-service: What is it and what does it mean for the IoT edge? Ericsson Blog.
<https://www.ericsson.com/en/blog/2019/12/tinyml-as-a-service-iotedge>
- Edge Impulse. (2024). Edge Impulse documentation: Machine learning for edge devices. <https://docs.edgeimpulse.com/>
- Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). 2015 Military Communications and Information Systems Conference (MilCIS), 1–6.
<https://doi.org/10.1109/MilCIS.2015.7348942>
- Ray, P. P. (2022). A review on TinyML: State-of-the-art and prospects. Journal of King Saud University — Computer and Information Sciences, 34(4), 1595–1623.
<https://doi.org/10.1016/j.jksuci.2021.11.019>
- Warden, P., & Situnayake, D. (2019). TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers. O'Reilly Media.